TECHNICAL DESIGN: MULTI-SOURCE CANDIDATE DATA TRANSFORMER

Author: Shreya Kumari • shreyasingh091102@gmail.com
Assignment: Eightfold Candidate Transformer

1. System Architecture & Pipeline

The system is built on a clean pipeline pattern using SOLID design principles. It decouples raw data ingestion, text extraction, data normalization, merge resolution, and runtime projection. The pipeline executes sequentially:

Input Sources → Detect Source → Parse/Extract → Normalize → Merge (Conflict Resolution) → Confidence Scoring → Projection Layer → Schema Validation → JSON Output

2. Canonical Schema Design

The canonical schema represents the single source of truth for candidate data. Defined using Pydantic models for validation, it maintains strict type constraints:

- **candidate_id:** Determinsitic unique identifier.
- **full_name / headline / location / years_experience:** Canonical single values.
- **emails / phones / links / skills:** Normalized arrays representing union of inputs.
- **experience / education:** Structured sub-arrays sorted descending by start date.
- **provenance:** Map of fields to their source file, extraction method, and raw values.
- **field_confidence / overall_confidence:** Calculated indicators.

3. Normalization Strategy

Normalization is executed on individual source records before merging to ensure consistency:

- **Phones:** Standardized to **E.164** format using the Google *phonenumbers* library.
- **Dates:** Transformed to **YYYY-MM** format. Relies on *python-dateutil* to handle formats ('Jan 2018', '2018/01', etc.). Resolves 'Present/Current' to 'Present' and defaults missing months to Jan.
- **Country:** Normalized to **ISO-3166 alpha-2** code by searching location tokens in a compiled dictionary.
- **Skills:** Maps synonyms ('C Plus Plus', 'cpp' → 'C++') using exact lookups and fuzzy *rapidfuzz* matching.

4. Merge & Conflict Resolution Strategy

Merging combines data from multiple inputs using a **deterministic precedence policy**. Candidate-authored primary sources (Resume PDF) overwrite third-party secondary sources (Recruiter CSV).

- **Precedence:** Resume PDF (Priority 2) > Recruiter CSV (Priority 1) > Unknown (0).
- **Conflict Resolution:** For single fields (e.g. name), the value with highest source priority wins. If a conflict occurs, the winning source and method are recorded in the provenance schema.
- **Arrays & Nesting:** Email/phone arrays are unioned and deduplicated. Work history and education sections are merged by fuzzy matching overlapping intervals (same company/degree + start year) and sorted chronologically.

5. Explainable Confidence Scoring

Confidence scores are computed deterministically per field based on source trustworthiness and extraction methodologies. Raw scoring assignments:

- **Resume (Direct):** 0.95 | **CSV (Direct):** 0.90
- **Regex Extraction:** 0.80 | **Heuristics:** 0.60 | **Unknown:** 0.30

The **overall_confidence** score is calculated as the mathematical average of all populated field-level confidences.

6. Provenance Model

To guarantee system explainability, the canonical output maps every populated field to its original provenance record: {field: {source: str, method: str, value: Any}}. If fields are merged from multiple sources, the winner's metadata is preserved, keeping auditability intact.

7. Configurable Runtime Projection Layer

Keeps the canonical profile separate from external outputs. Configured via config.json:

Runtime Config: Supports field inclusion, field renaming, normalization rules, provenance/confidence toggles, and configurable missing-value behavior without code changes.

- **Inclusion/Exclusion:** Specify output fields in include_fields list.
- **Field Renaming/Mapping:** Maps canonical terms to external targets (e.g., full_name → name).
- **Metadata Toggles:** Dynamically enable/disable confidence arrays and provenance tracking.
- **Missing Values:** Choose 'null' values or 'exclude' (omitting empty keys).

8. Schema Validation & Edge Cases

- **Validation:** Both the canonical profile and the final projected output are validated against Pydantic models. Validation errors format into readable messages detailing field location, expected type, and actual input.
- **Edge Cases:** Gracefully handles empty files, missing fields (resolves to null/omitted), and duplicate candidates (merged into one canonical record by checking email match prior to execution).

9. Assumptions & Future Improvements

- **Assumptions:** Input source files are accessible. PDFs contain extractable text (non-scanned). Profiles sharing an email address belong to the same candidate.
- **Future Enhancements:** Support additional parsers (LinkedIn, GitHub JSON, ATS XML). Integrate OCR (e.g. Tesseract) for scanned PDF resumes. Train ML models for semantic-based section parsing.